

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2023.0322000

# Scalable Hybrid Online Machine Learning for Real-Time Intrusion Detection in Electric Vehicle Charging Systems

JOSIAH KIPRUTO<sup>1</sup>, BERNARDI PRANGGONO<sup>1</sup> (Senior Member, IEEE), and RAJ SHUKLA<sup>1</sup>,

<sup>1</sup>School of Computing and Information Science, Anglia Ruskin University, Cambridge CB1 1PT, UK

Corresponding author: Bernardi Pranggono (e-mail: [bernardi.pranggono@aru.ac.uk](mailto:bernardi.pranggono@aru.ac.uk))

**ABSTRACT** Electric Vehicle Charging Systems (EVCS) are increasingly integrated into smart grids, but their openness and connectivity expose them to diverse cyber threats. Intrusion Detection Systems (IDS) are essential for securing these infrastructures; however, conventional offline models struggle with real-time data streams, evolving attack patterns, and concept drift. This study investigates online machine learning for real-time IDS in EVCS. Two approaches are evaluated: a baseline Adaptive Random Forest (ARF) with ADWIN drift detection, and a scalable hybrid ensemble integrating ARF with Naïve Bayes, k-Nearest Neighbors, and Logistic Regression under a meta-learning framework. Experimental results show that while the baseline ARF achieves competitive performance, it incurs high computational cost (267.98 minutes for the 15-class task). In contrast, the hybrid model reduces runtime to 81.65 minutes while improving accuracy (0.8582 vs. 0.8142), precision (0.8796 vs. 0.8341), recall (0.8582 vs. 0.8142), and F1-score (0.8344 vs. 0.8002). Notably, in binary classification, the hybrid ensemble achieves near-perfect detection ( $F1 = 1.0000$ ) while maintaining stability under drift-free conditions. These findings demonstrate that combining diverse classifiers enhances drift resilience, improves computational efficiency, and strengthens real-time detection, offering a scalable IDS solution tailored to EVCS and advancing the security and reliability of modern smart grids.

**INDEX TERMS** Electric Vehicle Charging Systems (EVCS), Intrusion Detection Systems (IDS), Online Machine Learning, Hybrid Ensemble Models, Concept Drift, Smart Grid Cybersecurity

## I. INTRODUCTION

The global shift towards electric mobility has resulted in substantial expansion in the adoption of electric vehicles (EVs), driven by the imperative to reduce carbon emissions, evolving energy policies, and advancements in technological innovation. This transition is accompanied by the rapid development of infrastructure systems, particularly Electric Vehicle Charging Systems (EVCS) (see Figure 1). According to research, the global EVCS industry is projected to expand from USD 15.57 billion in 2024 to USD 96.88 billion by 2032 [1]. The growth of the EV sector necessitates the implementation of robust, scalable, and secure systems, encompassing real-time communication, billing, and vehicle-grid integration functionalities [2].

Standardized communication protocols were developed to achieve interoperability between various stakeholders in the EV ecosystem (EVs, charging stations, and backend servers). The two protocols mainly used are Open Charge Point Protocol (OCPP) and ISO 15118, which provide key services (e.g.,

plug-and-charge, secure authentication, firmware updates, and remote monitoring) [3]. Although these protocols enable functionalities and integration of systems, they also create vulnerabilities for cyber attackers to take advantage of. The open and generally weakly secured communications channels open EVCS networks up to vulnerabilities like spoofing, port scans, SYN floods, and denial-of-service (DoS) attacks [4]. Cyber intrusions put user privacy and safety at risk, compromising the operational security of EV infrastructure and the larger smart grid [5].

This study advances the state of the art in EVCS cybersecurity by introducing a scalable online machine learning framework for real-time intrusion detection. The main contributions and their significance are summarized as follows:

- Drift-sensitive hybrid architecture: We propose a novel IDS framework that combines ARF with meta-learning, enabling dynamic adaptation to evolving attack patterns and concept drift in EVCS environments.
- Optimized base learner integration: By aggregating

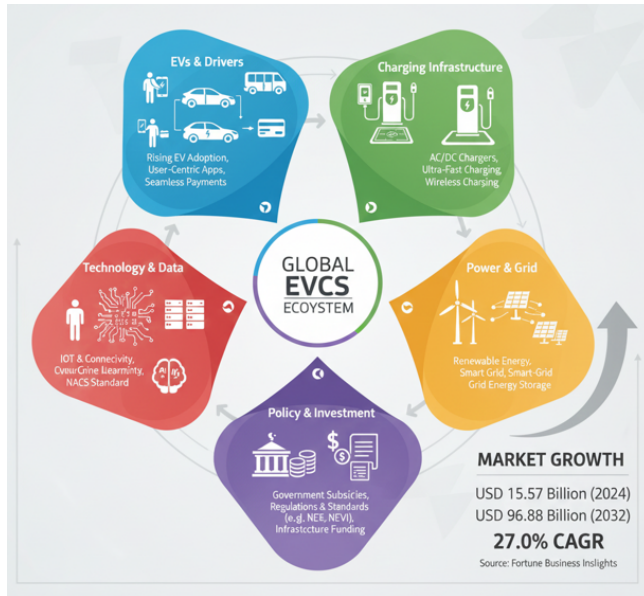


FIGURE 1: EVCS Ecosystems

Naïve Bayes, k-Nearest Neighbors, and Logistic Regression within the ensemble, the system balances detection accuracy, robustness, and computational efficiency.

- **Efficiency without performance loss:** We demonstrate that training time can be significantly reduced without degrading detection performance, a critical requirement for real-time deployment.
- **Modular streaming pipeline:** The study introduces a repeatable and extensible pipeline for online intrusion detection, which can be transferred to other cyber-physical systems requiring real-time security.
- **Comprehensive validation:** The model is empirically validated on the dataset across both binary and multi-class intrusion detection tasks, addressing a gap in IDS research and reinforcing its practical applicability.

These contributions collectively advance the state-of-the-art in adaptive cybersecurity for smart grid-integrated EV infrastructure and provide a foundation for future research in scalable, intelligent IDS solutions.

The remainder of this paper is structured as follows: Section II reviews related work on intrusion detection systems for EVCS. Section III describes the dataset, preprocessing methods, and experimental setup. Section IV presents the results and comparative analysis. Section V discusses the findings and their implications, and Section VI concludes the paper with recommendations for future research.

## II. RELATED WORK

Static or offline machine learning methods have been extensively applied in the development of intrusion detection systems (IDS). These models are typically trained on a pre-determined dataset prior to deployment and remain constant during their operational phase. Conventional algorithms, such as decision trees, support vector machines (SVMs), and

random forests, have demonstrated adequate performance in general network intrusion detection tasks. Within EVCS environments, these models have been utilized to identify known threats through the analysis of preprocessed traffic logs. Nevertheless, these models presuppose a stationary data distribution, wherein normal and malicious traffic patterns remain unchanged over time. This assumption is invalid in actual EVCS networks, where traffic dynamics are influenced by firmware updates, the introduction of new EV models, backend server upgrades, and variations in user behavior [6]. Consequently, static models become obsolete—or less effective—over time. This obsolescence necessitates periodic retraining, which is resource-intensive and impractical for implementation in real-time applications.

Online machine learning is an area of machine learning where the algorithm processes data continuously, one instance at a time or in small batches, instead of training on the data set all at once. Online learning is distinct from batch learning in that a batch learning model must be retrained when new data is acquired. In contrast, an online learner processes the newest data instance, essential for streaming data [4]. This is especially important in situations such as EVCS, where the network traffic evolves rapidly, and new attack patterns may develop unpredictably. Online learning allows for adaptive actions as events occur, and IDS can operate successfully in real-time, even under non-stationary data distribution and unknown threats.

One of the key advantages of online machine learning in EVCS is the ability to deliver low-latency updates with relatively little computational cost [7]. EVCS is also distributed over vast geographic areas, and cyber anomalies will need real-time decisions made to avoid interruptions. Static models may require whole-scale retraining on enormous datasets, which could lead to delays in detection and response. Online learners can process a single new data point nearly instantaneously, which makes them a good fit for detecting anomalies in real-time in EVCS traffic. Additionally, a significant advantage of online learning systems is that they are memory-efficient, which is essential to consider when employing resource-limited edge devices that will ultimately be part of the charging station.

Several online learning algorithms have been created and utilized successfully in IDS. One of the first algorithms is the Hoeffding Tree, or Very Fast Decision Tree (VFDT). It can construct a decision tree incrementally and applies continual checking of the Hoeffding bound to determine at any point that there is sufficient evidence to split the nodes, and hence is fast on large data streams. The ARF ensemble method also comprises Hoeffding Trees, which contribute to further classification accuracy. ARF is more resilient due to the application of drift detectors per tree in conjunction with a simple majority voting of the ensemble, rendering the method particularly useful in high-dimensional and dynamic settings of intrusion detection [8]. Other types of ensembling (Online Bagging and Online Boosting) apply conventional ideas of bagging and boosting to enhance generalization in

the situation of online learning.

A significant problem that has been addressed by online learning in IDS is the issue of concept drift (the idea that the statistical properties of the target variable change over time). In EVCS, there could be a drift from updated communication protocols, new additions of EV models, or updates to adaptive attack characteristics. Online IDS approaches typically include drift detection mechanisms that monitor the learning algorithm's performance and notify the algorithm that model adaption is required. ADWIN is a widely used drift detection method, which keeps a sliding window of variable size and uses statistical tests to observe changes in data distribution, updating the model or retraining based on the new distribution when drift is detected [9]. The Drift Detection Method (DDM) monitors error rates and raises an alarm when a statistically significant increase occurs. The Early Drift Detection Method (EDDM) extends DDM by looking at the distance between classification errors and is more applicable to gradual drifts.

The existing context shows that online ML offers some promise for IDS across different contexts, including smart grids, industrial control systems, and vehicular networks. For instance, Almanian et al. implemented online random forest classifiers on network traffic and noted improvements in detection accuracy and drift adaptation [10].

The development of advanced and adaptive IDS for deployment in EVCS has highlighted the issues associated with static, classic classifier systems. As threats evolve and data streams become increasingly variable, models can no longer rely on one-off classifications; they must also adapt to concept drift. Although DL approaches have made significant progress in offline environments, producing a model that requires tremendous processing power and cannot account for changes in a real-time, resource-depleted environment [11]. As a result, hybrid ensemble learning methods have started to receive attention for providing flexibility with performance in streaming data scenarios.

Existing work has extensively confirmed one of the benefits of the hybrid ensemble learners compared to one-model learners in situations of streaming and dynamic data. For instance, Montiel et al. confirmed that combining NB and tree-based learners substantially improved accuracy and reaction time when there is drift in online learning situations [12]. Bifet and Gavalda were among the first authors who highlighted the importance of adaptive learning mechanisms in data streams through their work on Adaptive Sliding Windows, offering the origins of dynamic drift detection in real-time systems [13].

Empirical evaluation using the CICEVSE2024 dataset revealed improved performance across all streaming metrics, including accuracy, precision, recall, and F1-score. This was especially evident in the 15-class classification task, where the hybrid model outperformed the baseline ARF setup. Additionally, execution time was significantly reduced, making the hybrid solution more practical for real-time deployment in EVCS. Environments. These results support earlier claims

in the literature and demonstrate that hybrid models (when thoughtfully integrated with meta-learning and drift-aware mechanisms) offer not only predictive accuracy but also computational efficiency under evolving network conditions.

Despite progress in online learning for IDS, research remains limited in applying hybrid ensemble models to real-time cybersecurity, particularly within EVCS. Most studies have focused on traditional ensemble models like ARF without integrating meta-learning for adaptive prediction weighting. Additionally, while concept drift is a known challenge in streaming environments, many approaches lack the integration of dynamic drift detection mechanisms such as ADWIN. This omission restricts the model's ability to maintain high accuracy under evolving network conditions. Furthermore, execution efficiency and real-time deployment feasibility are rarely addressed, even though they are critical for applications in fast-response domains like EVCS.

Another significant gap lies in the underutilization of cumulative metric monitoring alongside online learning. Few models track performance metrics such as accuracy, precision, recall, and F1-score over time, which are essential for evaluating adaptability in streaming data scenarios. The existing literature also lacks empirical comparisons between baseline ARF models and more advanced hybrid approaches using the same dataset for binary and multiclass tasks. As such, there is limited evidence on the superiority of hybrid models in identical experimental settings. These gaps highlight the need for a flexible, drift-aware, and performance-monitored hybrid IDS model, which is addressed in this study.

### III. METHODOLOGY

In this section, we explain the methodology adopted in this research.

#### A. DATASET DESCRIPTION AND PREPROCESSING

The CICEVSE2024 dataset [14], a formally defined intrusion detection dataset designed explicitly for EVCS, was utilized in this work. The dataset consisted of tabular records, where each row of instances contained a collection of engineered attributes, such as packet size, flow length, and protocol type, among others, and related labels representing the type of traffic. Two labelling schemes were employed: a 2-class binary classification (Benign vs. Attack) and a 15-class multiclass classification, reflecting different types of attacks, such as DoS, spoofing, and ransomware.

Before streaming analysis could be applied, some pre-processing had taken place. This encompassed handling of null values, one-hot encoding of the categorical values, and normalisation of numerical ranges of feature values. All of this helped make the data amenable to the algorithms used in online learning, which work incrementally on data. The Hoeffding Tree's feature importance analysis capability can facilitate utilization of feature priority based on information gain for quick and effective feature selection [15]. To simulate practical traffic, the work also converted the dataset into a stream format, which enables handling of online learning and

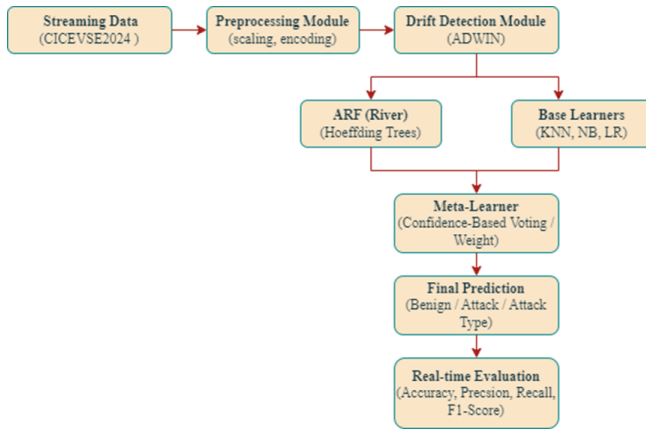


FIGURE 2: Hybrid Model Architecture

drift detection (an important necessity of real-time intrusion detection). Such clean stream data would compare equally well, allowing a comparison of both baseline and hybrid performance, objectively and fairly determining optimal performance in a dynamic network and live environment.

### B. HYBRID MODEL DESIGN

The hybrid framework had evolved into enhanced streaming classification platforms that utilized various learners in a mixed architecture of drift-aware ensembles. The foundation of the architecture was an ARF model, recognized for its practical application in processing dynamic data streams, as well as its in-built conceptual drift detectors. This base was placed on top of a set of dynamic base learners, including KNN, NB, and LR. All of the base learners were written using libraries that can be updated over the net, and therefore can exist non-destructively with the data stream.

A meta-learning structure was constructed that would align the decisions of the ensemble. This module is based on continuously approximating the performance of base learners on new data, using a weighting scheme that assigns likelihoods to predictions using new statistics of accuracy or confidence. Such weighting was applied to a confidence-weighted voting process, whereby predictions would be a function of the relative reliabilities of each learner within the current concept window. In situations where performance breakdowns were detected, the meta-learner would trigger the replacement or retraining of inadequately performing models based on drift detection feedback supplied by the base ARF components.

Fig. 2 illustrates the architecture of the proposed hybrid intrusion detection model developed in this study. It shows how streaming data is processed through preprocessing, drift detection, multiple base learners, and a meta-learning layer to generate adaptive and accurate real-time predictions.

### C. EXPERIMENTAL SETUP

All experiments were conducted on Google Colab Pro (70GB of RAM), providing sufficient computational power for real-time stream processing and model training. Necessary li-

braries included River for online learning, Scikit-learn for base learner integration, and Pandas for data handling [15]–[18].

A simulated data stream was created from the preprocessed CICEVSE2024 dataset to mimic the real-time arrival of data. Instances were fed sequentially to the models using fixed time intervals, preserving the original order of records to maintain the temporal structure critical for drift detection and streaming evaluation. Both binary (2-class) and multiclass (15-class) classification experiments were conducted in separate runs, ensuring consistent benchmarking across both scenarios.

The evaluation procedure was designed to capture key performance aspects. Real-time tracking of accuracy, F1-score, recall, and precision was enabled via cumulative metric monitoring. Concept drift checkpoints were integrated to observe model adaptability over time. Execution time was automatically recorded for each classification scenario to assist with an efficiency analysis. Random seeds were frozen when initializing the models to ensure reproducibility. All configuration files, model versions, and experiment scripts were version-controlled with Git. The test conditions, including the batch size, the order of streams presented to the model, and all model parameters, remained consistent across replications to provide a valid comparative analysis of the baseline and hybrid approaches. This methodological framework enabled repeatable measurements of the performance, robustness, and scalability of the proposed method.

### D. EVALUATION METRICS AND MONITORING

The model's performance was monitored through four key real-time metrics: Accuracy, Precision, Recall, and F1-Score. This selective choice of metrics was due to their direct applicability in estimating classification performance, particularly in streaming intrusion detection situations where class imbalance and drift are known to affect outcomes significantly. Accuracy estimated the general correctness of the prediction, while Precision and Recall provided information on the model's capability to detect the minority class (attack). F1-Score, which is the harmonic mean of Precision and Recall, offered a balanced estimation, which is of particular significance in situations of different data distributions.

To monitor such metrics effectively throughout the entire data stream, we employed a cumulative tracking system via River's built-in metrics module. Performance metrics were computed at each time step using a sliding window functionality, which facilitated real-time smoothing of trends and prevented abrupt jumps in metrics due to transient drift or noise. This window-based system of monitoring facilitated continuous observation of model performance during variable data fragments and allowed for a response to declines in performance.

Matplotlib was used to visualize the results of the metric, allowing the development of the metric over time to be observed [19]. This visualization helped verify the workability of the drift handling and adaptive learning components, both in binary and multiclass scenarios, presenting clear evidence

of real-time IDS activity within the temporal window of analysis.

### E. VISUALIZATION AND INTERPRETABILITY

To make the models more comprehensible and easier to interpret, and to facilitate better estimation of their performance, various visualization tools were utilized during the experiments. Visualization of measures, such as cumulative accuracy and F1-score over time, enabled tracking of the models' learning in real-time. Similarly, visualizations of drift showed the identified points of drift and model resets, helping to understand the adaptive learning aspect of the learners. Feature importance visualization was achieved by observing the decision path of Hoeffding Trees [20], alongside the accumulated feature value of the top N contributing features over time. Visualization helped interpret how decisions were made and which features contributed the most to the prediction, ultimately leading to a better, more transparent, and more interpretable IDS.

### F. PERFORMANCE METRICS

To effectively assess the performance of an ML model, it is crucial to employ an appropriate metric that reflects the accuracy and reliability of the model. For binary classification tasks, such as distinguishing between two outcomes, the confusion matrix is a commonly used, straightforward metric. This matrix provides four distinct prediction outcomes: (i) True Positive, where a fraudulent transaction is accurately identified as fraud; (ii) False Positive, where a legitimate transaction is mistakenly labeled as fraud; (iii) False Negative, where a fraudulent transaction is incorrectly marked as legitimate; and (iv) True Negative, where a legitimate transaction is correctly classified as such. A multi-pronged metric approach was employed to ascertain model performance, utilizing the validation dataset:

- Accuracy reflects the proportion of correct predictions made.
- Precision quantifies the accuracy of positive predictions.
- Recall highlights the fraction of positives that were rightly classified.
- F1-Score offers a harmonic balance between precision and recall, particularly crucial when faced with class imbalances.

## IV. EVALUATION AND RESULTS

In this section, the performance metrics of the proposed system were evaluated.

### A. PROPOSED HYBRID MODEL (HT + NB + LR + KNN WITH ADWIN)

#### 15-Class Problem

The proposed hybrid model was evaluated on the 15-class problem, and the results demonstrate competitive performance with effective adaptation to concept drifts. A total of 17 drift events were detected across the stream at varying

TABLE 1: Table 4.5: Drift Detection Events for the 15-Class Problem

| Drift Event | Instance Detected | Duration (minutes) |
|-------------|-------------------|--------------------|
| 1           | 480               | 0.02               |
| 2           | 10,080            | 0.74               |
| 3           | 17,664            | 1.28               |
| 4           | 47,488            | 3.44               |
| 5           | 59,168            | 4.22               |
| 6           | 72,928            | 5.15               |
| 7           | 73,760            | 5.21               |
| 8           | 85,824            | 6.03               |
| 9           | 119,808           | 8.18               |
| 10          | 252,608           | 16.84              |
| 11          | 453,344           | 29.41              |
| 12          | 514,112           | 33.23              |
| 13          | 654,912           | 42.11              |
| 14          | 662,464           | 42.58              |
| 15          | 699,232           | 44.92              |

TABLE 2: Table 4.6: Performance Summary for the 15-Class Problem

| Metric         | Value         |
|----------------|---------------|
| Execution Time | 81.65 minutes |
| Drift Events   | 17            |
| Final Accuracy | 0.8582        |
| Precision      | 0.8796        |
| Recall         | 0.8582        |

intervals, beginning as early as instance 480 (0.02 minutes) and continuing up to instance 852,064 (54.54 minutes). This indicates the sensitivity of the model to evolving data distributions, particularly in the early stages of stream processing.

The hybrid model achieved a final accuracy of 0.8582, with a precision of 0.8796, recall of 0.8582, and an F1-score of 0.8344, confirming its ability to balance predictive performance across multiple classes. Importantly, the execution time was reduced to 81.65 minutes, which is significantly more efficient than the baseline model, highlighting the computational benefits of the hybrid approach.

Figures 3 to 9 illustrate the evaluation metrics over time, drift detection events, class distribution dynamics, and feature importance. The metrics plots (Figures 3 and 4) show that accuracy, precision, recall, and F1-score improved gradually after initial drifts, stabilizing around higher values. The processing time distribution (Figure 5) indicates consistent runtime efficiency, with no significant outliers detected. Drift and metrics contributions are visualized in Figure 6, confirming the interplay between adaptation and performance maintenance. Furthermore, Figure 7 highlights accuracy variations before and after drift events, showing that the model consistently recovered performance after drift adaptation.

The class distribution plot (Figure 8) reveals the imbalance inherent in the 15-class dataset, with some classes dominating the stream. Finally, feature importance (Figure 9) highlights the most influential predictors, with variables such as `dst2sr_bytes`, `bidirectional_max_ps`, and `application_is_guessed` emerging as key drivers of classification performance.

Overall, the results indicate that the hybrid model offers a

strong balance between accuracy, adaptability, and computational efficiency for the 15-class problem, outperforming the baseline in terms of drift adaptation and execution time.

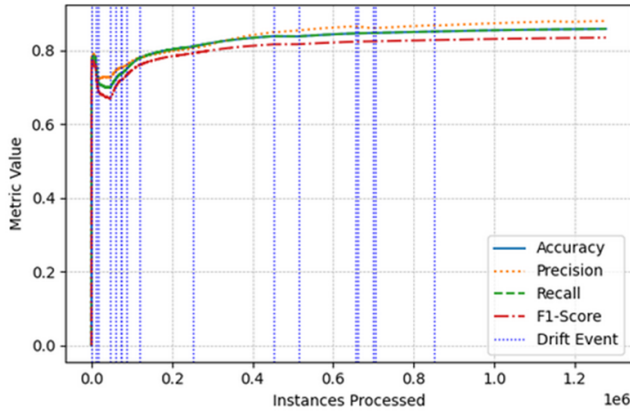


FIGURE 3: Performance metrics with drift events over 15-class stream

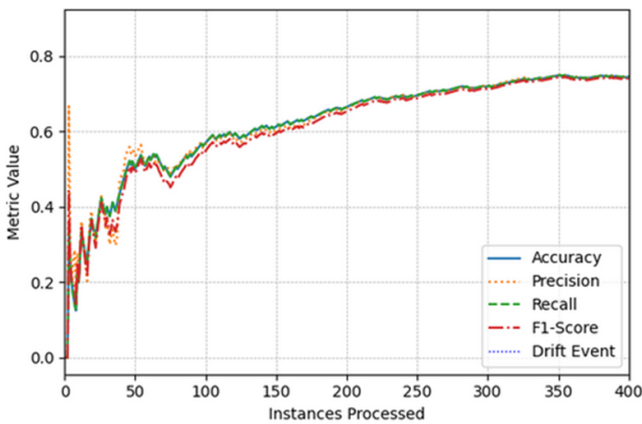


FIGURE 4: Performance metrics over the first 400 instances of the 15-class stream

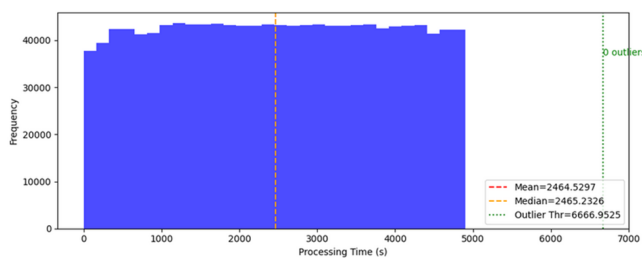


FIGURE 5: Processing time distribution for 15-class problem

## 2-Class Problem

The hybrid model achieved exceptional performance in the binary classification problem. Unlike the 15-class case, no concept drifts were detected throughout the entire data stream, indicating the robustness and stability of the model to

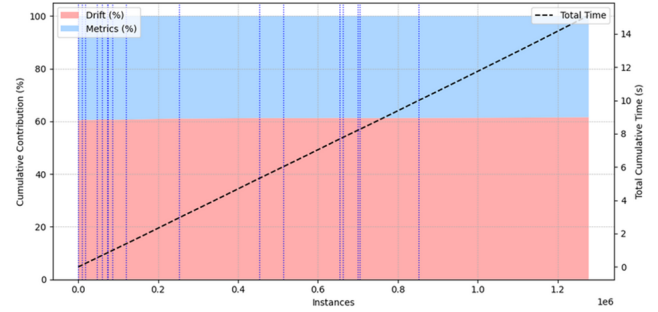


FIGURE 6: Cumulative contributions of drift detection and metrics evaluation with total time

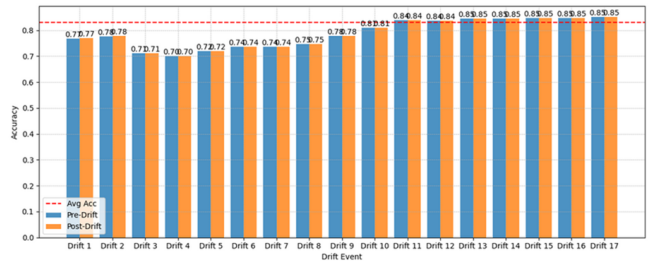


FIGURE 7: Pre- and post-drift accuracy comparison across 17 drift events (15-class)

distinguish between benign and attack traffic. This stability translated into near-perfect metrics, with a final accuracy of 0.9999, precision of 0.9999, recall of 1.0000, and an F1-score of 1.0000.

In addition to outstanding predictive performance, the model completed execution in 70.92 minutes, a significant improvement in computational efficiency compared to both the baseline and the 15-class hybrid experiment. Overall, the hybrid model shows good generalizability in the binary condition and is more accurate and less sensitive to drift.

TABLE 3: Table 4.7: Performance Summary – Hybrid Model (2-Class Problem)

| Metric                   | Value  |
|--------------------------|--------|
| Accuracy                 | 0.9999 |
| Precision                | 0.9999 |
| Recall                   | 1.0000 |
| F1-Score                 | 1.0000 |
| Execution Time (minutes) | 70.92  |
| Drifts Detected          | 0      |

The performance trends are further supported by Figures 10 to 15, which illustrate the model's metrics, processing time, and feature importance. Figures 10 and 11 show how accuracy, precision, recall, and F1-score remain consistently high across the stream, with no deviations caused by drifts. Figure 12 highlights the stable distribution of processing times, while Figure 13 illustrates the cumulative contribution of drift and metrics computations, confirming that drift monitoring required negligible overhead.

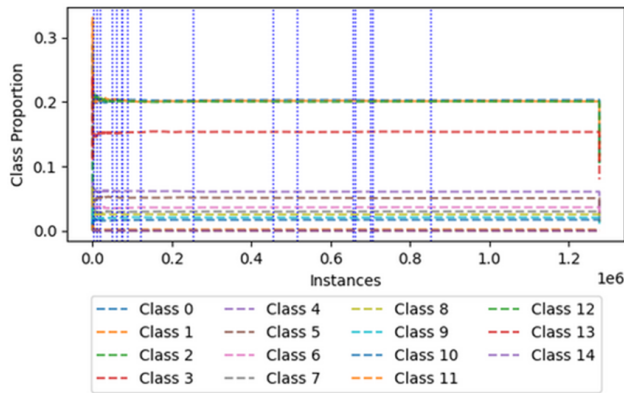


FIGURE 8: Class proportion distribution across instances for 15-class problem

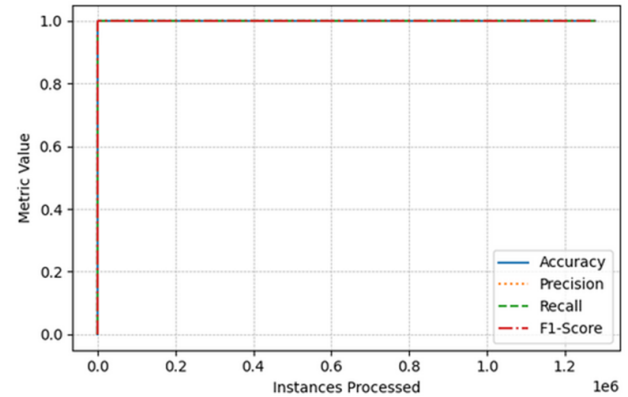


FIGURE 10: Performance metrics for 2-class problem with hybrid model

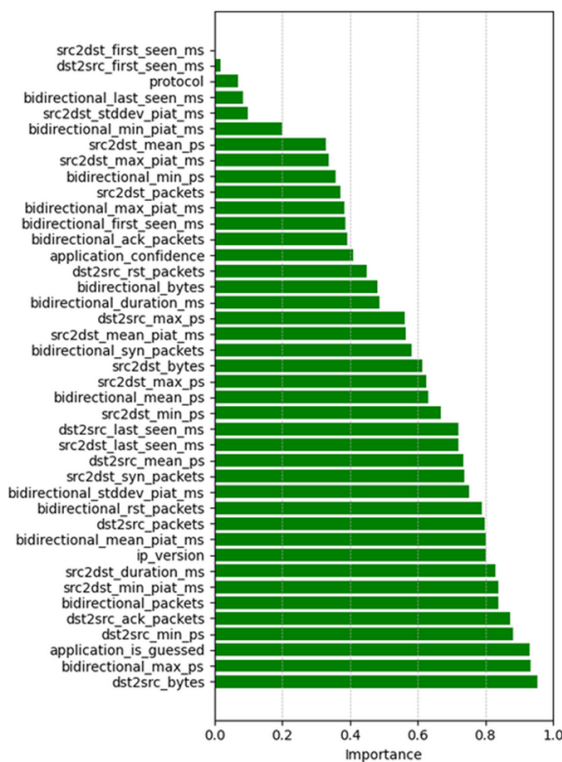


FIGURE 9: Feature importance ranking for the hybrid model on 15-class stream

The class distribution plot Fig. 14 shows the clear dominance of Class 1 over Class 0 in the dataset, yet the model effectively handled this imbalance to maintain perfect classification. Finally, the feature importance plot (Figure 15) indicates that variables such as `dst2src_min_ps`, `src2dst_bytes`, `ip_version`, and `bidirectional_max_ps` were among the most influential features driving the classification decisions.

Overall, the hybrid model demonstrated near-perfect classification ability, zero drift events, and high efficiency in the 2-class case, reinforcing its suitability for practical deploy-

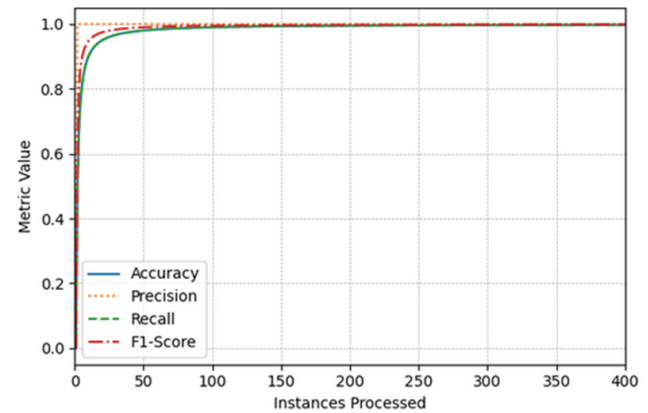


FIGURE 11: Early-stage performance evolution across first 400 instances for 2-class hybrid model

ment in intrusion detection scenarios with binary outcomes.

## B. COMPARATIVE ANALYSIS

To assess the effectiveness of the proposed hybrid model, we conducted a comparative analysis against the baseline approach [7] across both the 15-class and 2-class problems. The comparison considered execution time, final accuracy, precision, recall, F1-score, and the number of drift detections.

Key Improvements Observed:

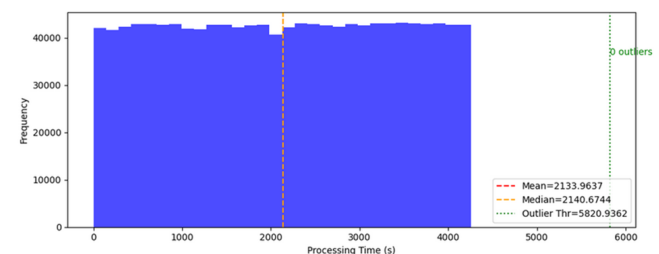


FIGURE 12: Processing time distribution for 2-class hybrid problem

TABLE 4: Summary Comparison of Baseline vs Hybrid Model

| Task     | Model    | Execution Time (min) | Accuracy | Precision | Recall | F1-Score |
|----------|----------|----------------------|----------|-----------|--------|----------|
| 15-Class | Baseline | 267.98               | 0.8142   | 0.8341    | 0.8142 | 0.8002   |
|          | Hybrid   | 81.65                | 0.8582   | 0.8796    | 0.8582 | 0.8344   |
| 2-Class  | Baseline | 106.54               | 0.9913   | 0.9999    | 0.9914 | 0.9956   |
|          | Hybrid   | 70.92                | 0.9999   | 0.9999    | 1.0000 | 1.0000   |

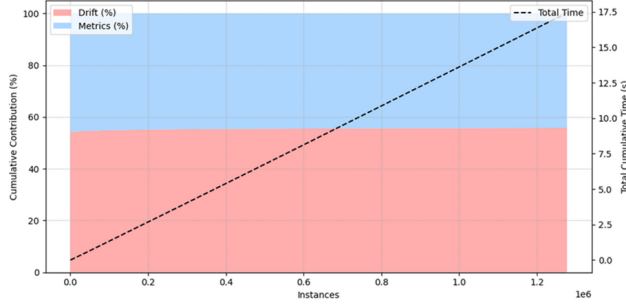


FIGURE 13: Cumulative contribution of drift detection and metrics evaluation with total time (2-class)

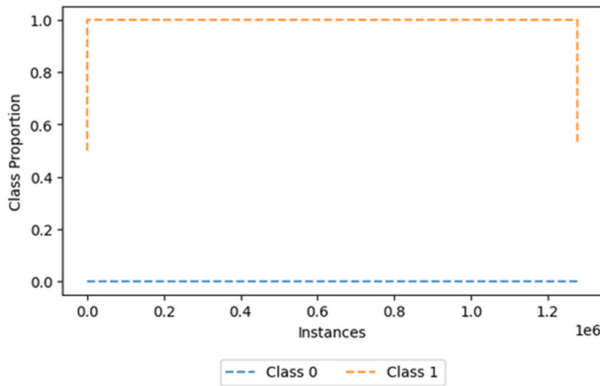


FIGURE 14: Class proportion distribution for 2-class dataset under hybrid model

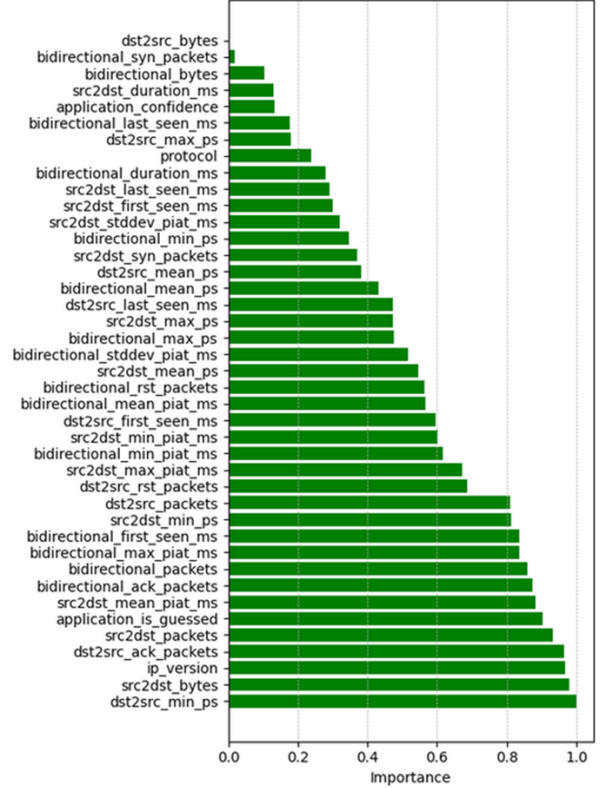


FIGURE 15: Feature importance ranking for 2-class hybrid model

distributions.

The comparison results demonstrated the effectiveness, precision, and reliability of the hybrid model as a more efficient means for real-time intrusion detection in contrast to the baseline method.

## V. CONCLUSION

The overarching goal of this study was to enhance real-time intrusion detection capabilities in EVCS. Specifically, this study aimed to investigate the feasibility of applying and exploiting adaptive online learning methods given streaming data related to cybersecurity. The research followed a two-phase methodology. The first involved replicating a baseline model using ARF based on prior literature to establish a reliable performance benchmark. The second phase focused on developing a novel hybrid model that combined ARF with dynamic base learners and a meta-learning strategy to enhance decision-making under drift and class imbalance

TABLE 5: Performance Comparison

| Authors                  | Year | Dataset                        | Model                                  | [h]<br>Learning Method | Class | Accuracy | Precision | Recall | F1-Score |
|--------------------------|------|--------------------------------|----------------------------------------|------------------------|-------|----------|-----------|--------|----------|
| Buedi et al. [14]        | 2024 | CICEVSE2024<br>Network Traffic | Random Forest                          | offline                | 15    | 0.937    | 0.969     | 0.937  | 0.947    |
|                          |      |                                | Support Vector Machine                 | offline                | 15    | 0.941    | 0.915     | 0.941  | 0.927    |
| Bozdemiroglu et al. [21] | 2024 | CICEVSE2024<br>Network Traffic | Decision Tree                          | offline                | 13    | 0.9999   | 0.9999    | 0.9999 | 0.9999   |
| Purohit et al. [22]      | 2024 | CICEVSE2024<br>Network Traffic | Federated Learning                     | federated              | 2     | 0.9697   | -         | -      | 0.9740   |
| Rahman et al. [23]       | 2025 | CICEVSE2024<br>Host Data       | Deep Learning                          | federated              | 3     | 0.9754   | 0.9757    | 0.9754 | 0.9754   |
| Benfarhat et al. [24]    | 2025 | CICEVSE2024<br>Host Data       | Temporal CNN                           | offline                | 2     | 1.0      | -         | -      | -        |
|                          |      |                                |                                        |                        | 5     | 1.0      | -         | -      | -        |
|                          |      |                                |                                        |                        | 17    | 0.9999   | -         | -      | -        |
| Makhmudov et al. [7]     | 2025 | CICEVSE2024<br>Network Traffic | Adaptive Random Forest                 | online                 | 2     | 0.9931   | 0.9999    | 0.9914 | 0.9956   |
|                          |      |                                |                                        |                        | 15    | 0.9840   | 0.9840    | 0.9840 | 0.9831   |
| Our Proposed Model       | 2025 | CICEVSE2024<br>Network Traffic | Hybrid (HT + NB + LR + KNN with ADWIN) | online                 | 2     | 0.9999   | 0.9999    | 1.0    | 1.0      |
|                          |      |                                |                                        | online                 | 15    | 0.8582   | 0.8796    | 0.8582 | 0.8344   |

conditions.

Throughout the project, emphasis was placed on the challenges of concept drift and the demands of real-time processing. To model realistic EVCS world environment simulations, data was streamed incrementally without batch re-training with updates from within the model's perspective. It was evaluated against two tasks: a low-complexity binary classification task (benign or attack) and a higher-complexity 15-class multiclass task mirroring network attack categories. This two-task evaluation methodology provided informative insights into model behaviour, allowing us to analyse model behaviour across a suite of classification complexity and drift scenarios. We did not conduct our research upon offline static evaluations; we wished to conduct more realistic streamed-based studies that were indicative of the deployed operating context within EVCS operational systems.

## REFERENCES

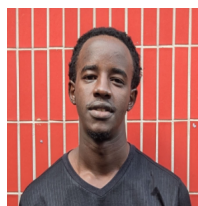
- [1] Fortune Business Insights, "Electric Vehicle Charging Station Market Size, Forecast [2032] - Fortune Business Insights," <https://www.fortunebusinessinsights.com/electric-vehicle-ev-charging-stations-market-102058>, 2025, accessed: 2025-12-09.
- [2] A. Kaur, N. Valizadeh, D. N. Jha, T. Szydio, J. RK Rajasekaran, V. Kumar, M. Barika, J. Liang, R. Ranjan, and O. Rana, "Cybersecurity challenges in the ev charging ecosystem," *ACM Computing Surveys*, vol. 58, no. 1, pp. 1–32, 2025.
- [3] Z. Garofalaki, D. Kosmanos, S. Moschyiannis, D. Kallergis, and C. Douligeris, "Electric vehicle charging: A survey on the security issues and challenges of the open charge point protocol (ocpp)," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 3, pp. 1504–1533, 2022.
- [4] T. Aljohani and A. Almutairi, "A comprehensive survey of cyberattacks on evs: Research domains, attacks, defensive mechanisms, and verification methods," *Defence Technology*, vol. 42, pp. 31–58, 2024.
- [5] S. Hijgenaar, A. Ştefanov, A. M. Van Voorden, and P. Palensky, "Cyber resilience of electric vehicle charging in smart grids: the dutch case," *IEEE Access*, 2025.
- [6] K. Sareddine, M. A. Sayed, C. Assi, R. Atallah, S. Torabi, J. Khoury, M. S. Pour, and E. Bou-Harb, "Ev charging infrastructure discovery to contextualize its deployment security," *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 1287–1301, 2023.
- [7] F. Makhmudov, D. Kilichev, U. Giyosov, and F. Akhmedov, "Online machine learning for intrusion detection in electric vehicle charging systems," *Mathematics*, vol. 13, no. 5, p. 712, 2025.
- [8] M. A. Shyaa, N. F. Ibrahim, Z. Zainol, R. Abdullah, M. Anbar, and L. Alzubaidi, "Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems," *Engineering Applications of Artificial Intelligence*, vol. 137, p. 109143, 2024.
- [9] M. H. Bahar and H. N. Saad, "Adaptive windowing (adwin3) to learning from time-changing data stream," in *International Conference on Intelligent Systems Design and Applications*. Springer, 2023, pp. 150–163.
- [10] M. Almania, A. Zainal, F. A. Ghaleb, A. Alnawasrah, and M. Al Qerom, "Adaptive intrusion detection system with ensemble classifiers for handling imbalanced datasets and dynamic network traffic," *Journal of Robotics and Control (JRC)*, vol. 6, no. 1, pp. 114–123, 2025.
- [11] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM computing surveys (CSUR)*, vol. 46, no. 4, pp. 1–37, 2014.
- [12] J. Montiel, J. Read, A. Bifet, and T. Abdesslem, "Scikit-multiflow: A multi-output streaming framework," *Journal of Machine Learning Research*, vol. 19, no. 72, pp. 1–5, 2018.
- [13] A. Bifet, R. Gavalda, G. Holmes, and B. Pfahringer, *Machine learning for data streams: with practical examples in MOA*. MIT press, 2023.
- [14] E. D. Buedi, A. A. Ghorbani, S. Dadkhah, and R. L. Ferreira, "Enhancing ev charging station security using a multi-dimensional dataset: Cicevse2024," in *IFIP Annual Conference on Data and Applications Security and Privacy*. Springer, 2024, pp. 171–190.

- [15] Google, "Google colabory," <https://colab.research.google.com>, 2023, accessed: 2025-01.
- [16] J. Montiel, J. Read, A. Bifet, and T. Abdesslem, "River: machine learning for streaming data in python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.
- [17] F. Pedregosa, G. Varoquaux, A. Gramfort et al., "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [18] W. McKinney, "Data structures for statistical computing in python," *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010.
- [19] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [20] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2000, pp. 71–80.
- [21] H. Bozömeroğlu and Z. Gürkaş-Aydın, "Elektrikli araç şarj istasyonlarının siber güvenliği: Cic-evse 2024 veri kümesi ile ids sistemlerinin performans analizi," in *2024 17th International Conference on Information Security and Cryptology (ISCTürkiye)*. IEEE, 2024, pp. 1–6.
- [22] S. Purohit and M. Govindarasu, "FI-evcs: Federated learning based anomaly detection for ev charging ecosystem," in *2024 33rd International Conference on Computer Communications and Networks (ICCCN)*. IEEE, 2024, pp. 1–9.
- [23] M. M. Rahman, M. M. H. Chayan, K. Mehrin, A. Sultana, and M. M. Hamed, "Explainable deep learning for cyber attack detection in electric vehicle charging stations," in *Proceedings of the 11th international conference on networking, systems, and security*, 2024, pp. 1–7.
- [24] I. Benfarhat, V. T. Goh, C. L. Siow, M. Sheraz, and T. C. Chuah, "Temporal convolutional network approach to secure open charge point protocol (ocpp) in electric vehicle charging," *IEEE Access*, 2025.



**RAJ MANI SHUKLA** is a Senior Lecturer ( Assistant Professor) at Anglia Ruskin University, Cambridge, UK. His research intersects with AI, the Internet of Things, and cybersecurity. Some of the research topics that Raj worked on include: trustworthy and responsible AI, privacy-preserving AI, sustainable cities, healthcare data analytics, cybersecurity, and cloud/edge computing. Before joining ARU, Raj worked as a KTP Associate - Data Scientist at the University of Bristol. He completed his Ph.D. in Computer Science and Engineering at the University of Nevada, Reno, under the supervision of Dr. Shamik Sengupta. Raj worked as a Junior Research Fellow (JRF) at IIT Kanpur. He received master's degree from NIT Kurukshetra and completed B.Tech from BIET Jhansi.

...



**JOSIAH KIPRUTO** received an M.Sc. in Cyber Security with Distinction from Anglia Ruskin University, Cambridge, UK. His research interests include cybersecurity, critical infrastructure, and AI.



**BERNARDI PRANGGONO** (Senior Member, IEEE) is an Associate Professor in Cyber Security and Computer Networks at the School of Computing and Information Science, Anglia Ruskin University, Cambridge, UK. Dr. Pranggono received his B.Eng. degree in Electronics and Telecommunication Engineering from Waseda University, Japan, an M.DigComms degree in Digital Communications from Monash University, Australia, and a Ph.D. degree in Electronics and Electrical Engineering from the University of Leeds, UK. He has previously held academic and research positions at Sheffield Hallam University, Glasgow Caledonian University, Queen's University Belfast, and the University of Leeds. He has held industrial positions at Pricewaterhouse Coopers, Accenture, and Telstra. His current research interests include cybersecurity, the Internet of Things, AI, and green ICT.